

AutoCleanAI — Product Requirements Document

AI-Assisted Data Quality Analysis, Transformation & Validation Platform

Contents

0. Product Context	2
1. Target Users	3
2. Primary User Journey	3
3. Example User Story	3
4. Product Goals	4
5. Non-Goals	4
6. Product Success Metrics	4
7. Scale & Performance Requirements	6
8. Cost Requirements	6
9. Competitive / Alternative Analysis	6
10. Product Differentiation	8
11. Core Architecture	8
12. Authority Boundary	9
13. AI Failure Handling	9
14. AI Response Validation	10
15. Data Privacy Architecture	10
16. Sensitive Data Handling	10
17. Access Control & Roles	10
18. Cloud Security	11
19. Approval Workflow & SLA	11
20. Candidate Selection (Deterministic, Not a Second LLM Call)	11
21. Unnecessary Modification Rate	12
22. KEEP / FLAG / MODIFY	12
23. No-Change Final State	12
24. Retry Mechanism and Termination	12
25. Downstream Contract (Pipeline Mode)	12
26. Audit Immutability, Versioning, Compatibility, Rollback	13
27. Quality Vector & Distribution Preservation	13
28. Research Evaluation	13
29. Research Metrics	13
30. Real-World Evaluation	14
31. User Interface Requirements	14
32. Deployment Modes	14
33. Rollout Strategy	14
34. Development Timeline	14
35. Major Risks	15
36. Final Definition of Done	16
37. Final Product Philosophy	17

Product: AutoCleanAI **Version:** 6.0 **Status:** Pre-Implementation **Type:** AI-assisted data-

quality and transformation platform **Primary users:** Data Engineers, Data Analysts, ML Engineers, Data Platform Teams **Initial deployment:** Local + AWS **Primary data engine:** Polars **LLM:** Provider-agnostic, initially AWS Bedrock, single-agent **MVP input:** CSV **Core principle:** AI reasons and proposes → policy constrains → deterministic system executes → independent validator evaluates

Changelog from v5.0: Replaced the multi-agent specialist/Judge architecture with a **single-agent** architecture as the primary shipped design, based on cost, latency, and current research evidence (Section 9.4) that multi-agent debate/handoff does not reliably improve — and can degrade — LLM-generated data-cleaning proposals, while a single agent reasoning over the full evidence package is materially cheaper and faster to run at scale. Multi-agent decomposition is retained only as an optional, explicitly-tested research variant (System C in the evaluation framework), not as the default production path. All deterministic infrastructure (Policy Engine, IR, Compiler, Sandbox, Validator, provenance, audit) is unchanged, since none of it depended on how many LLM calls proposed the candidates.

0. Product Context

0.1 Problem

Data cleaning is often a combination of manual inspection, SQL/Python scripts, fixed validation rules, statistical analysis, domain knowledge, and repeated trial-and-error. The problem becomes significantly harder when datasets are large, messy, inconsistently formatted, or contain ambiguous problems.

Existing deterministic tools are excellent at detecting known violations, but they generally do not reason about ambiguous remediation. LLMs can reason about possible solutions, but unrestricted LLM-generated cleaning introduces hallucinations, incorrect assumptions, unsafe code, silent data loss, irreproducibility, and difficulty validating semantic correctness.

AutoCleanAI combines the strengths of both: a single reasoning agent proposes remediations grounded in deterministic evidence, and a fully deterministic pipeline constrains, executes, and validates whatever it proposes.

0.2 Why Single-Agent, Not Multi-Agent

This is a deliberate architectural choice, not a simplification made for convenience, and it should be read alongside Section 9.4.

- **Cost and latency.** A multi-agent pipeline (N specialist agents + a Judge) multiplies LLM calls per issue. At MVP scale (Section 8) this materially increases cost-per-dataset and job duration for uncertain benefit.
- **Research evidence.** Recent controlled studies on multi-agent debate applied to data cleaning found that a second agent reviewing or arbitrating another agent’s proposal can *degrade* the proposal (through critique-induced confusion) even while a second agent *does* help at detection-type tasks. AutoCleanAI’s architecture already isolates detection (deterministic profiler + issue detector) from generation (the single agent), so it captures the part of multi-agent value that the research supports without the part that doesn’t.
- **Simplicity of the safety story.** With one proposing agent, the “AI proposes, deterministic system decides” principle (Section 12) has one clean handoff point instead of several agent-to-agent handoffs, each of which is a place where errors, sycophancy, or drift could otherwise compound.
- **This is a hypothesis, tested, not assumed.** Multi-agent decomposition remains a first-class variant in the evaluation framework (Section 28) so the team can measure

whether it earns its cost on this specific task, rather than defaulting to it because it is architecturally interesting.

1. Target Users

1.1 Primary — Data Engineer

Problem: A data engineer receives a large raw dataset and needs to understand: *“What’s wrong with this data, what should I do about it, and can I prove what changed?”*

Desired outcome: Upload dataset → receive a data-quality report, detected issues, proposed solutions, validation results, clean dataset, and complete transformation history.

1.2 Secondary — Data Analyst

Wants to avoid spending hours manually identifying nulls, invalid values, duplicates, category inconsistencies, and basic outliers. Primarily wants a trustworthy cleaned dataset without losing control over what was changed.

1.3 Secondary — ML Engineer

Wants raw dataset → quality analysis → cleaning → validated dataset → ML pipeline, with confidence that preprocessing hasn’t introduced unexpected distribution changes.

1.4 Future — Data Platform Team

Could eventually integrate AutoCleanAI into ETL pipelines, data warehouses, feature pipelines, batch processing, and data governance systems.

2. Primary User Journey

Assisted: Upload Dataset → Automatic EDA → Issues Detected → Review Proposed Actions → AutoCleanAI Executes → Validation → Final Dataset → Audit / Download / Integrate.

Autonomous: Upload → Analyze → Clean → Validate → Download, while still retaining the complete audit trail.

3. Example User Story

A data engineer uploads `customer_data.csv`. AutoCleanAI discovers 8.3% missing income, 0.7% invalid age, 14 inconsistent country representations, and 1,284 exact duplicate records. The single reasoning agent proposes group-wise median imputation for income, flagging for invalid age, normalization for country, and removal for exact duplicates. The engineer sees 4 issues detected, 3 transformations accepted, 1 unresolved, and can inspect every decision before exporting the final dataset.

4. Product Goals

1. Automatically profile datasets.
 2. Detect common quality problems.
 3. Generate plausible remediation strategies via a single reasoning agent, grounded in deterministic evidence.
 4. Execute only controlled transformations.
 5. Validate transformations independently of the proposing agent.
 6. Minimize unnecessary modification.
 7. Preserve complete provenance.
 8. Safely refuse uncertain transformations.
 9. Reduce manual data-cleaning effort.
 10. Provide measurable evidence of where single-agent reasoning is sufficient and where it is not, rather than assuming an architecture in advance.
-

5. Non-Goals

AutoCleanAI is not initially intended to be:

- A general-purpose ETL platform
 - A data warehouse
 - A feature-engineering platform
 - An arbitrary Python execution environment
 - A replacement for data governance systems
 - A fully autonomous system for sensitive/high-risk transformations
 - A general-purpose ML model-training platform
 - A multi-agent debate or consensus system by default
-

6. Product Success Metrics

Each metric is marked **[Gating]** (required for MVP sign-off, Section 37) or **[Aspirational]** (tracked and reported, not a launch blocker).

6.1 User productivity — Aspirational for MVP, gating for GA

Cleaning time reduction: manual cleaning time vs. AutoCleanAI-assisted cleaning time. Target: $\geq 50\%$ reduction in time spent on supported cleaning tasks.

6.2 User acceptance — Aspirational

Percentage of proposed transformations users accept without modification. Target: $\geq 70\%$ for low/medium-risk supported problems. Not interpreted as correctness by itself — see Section 6.8.

6.3 Unnecessary Modification Rate (UMR) — Gating

Percentage of originally correct values/records unnecessarily modified. Target: $< 5\%$ on benchmark datasets where ground truth exists.

6.4 Unsafe transformation rejection — Gating

Percentage of transformations violating explicit safety constraints that are correctly rejected. Target: 100% for defined hard policy constraints in the benchmark environment.

6.5 Data retention — Gating

Avoid unnecessary row/column loss. Exact threshold depends on transformation type, defined per-operation in the Policy Engine configuration.

6.6 Cost — Gating (reporting requirement)

Every job reports LLM calls, tokens, execution time, compute time, and estimated cloud cost. Because the architecture is single-agent by default, cost-per-job is expected to be materially lower and more predictable than a multi-agent equivalent; this is itself tracked as a baseline for comparison if multi-agent variants are trialed later (Section 28).

6.7 Reliability — Gating

≥99% of jobs terminate in a defined state (ACCEPTED, REJECTED, UNRESOLVED, FAILED). No job should silently disappear or leave an ambiguous state.

6.8 Acceptance Rate vs. UMR — Explicit Tradeoff Policy

UMR is the gating metric. If a configuration change would raise acceptance rate but push UMR above 5% on benchmark datasets, the change is rejected regardless of acceptance-rate improvement. Acceptance rate is diagnostic, telling the team whether the system is too conservative or well-calibrated — it never justifies loosening the UMR ceiling.

6.9 Measurement Plan

Metric	Data source	Sample	Method
Time reduction (6.1)	User study	≥8 users, ≥5 datasets each	Timed manual vs. assisted cleaning on matched tasks; paired comparison
Acceptance rate (6.2)	Beta usage logs	All MVP beta jobs, min. 200 transformations	Accept-without-edit / total proposed, per risk tier
UMR (6.3)	Benchmark datasets	≥5 datasets with ground truth	Automated comparison against ground truth (Section 29)
Unsafe rejection (6.4)	Adversarial/synthetic test suite	≥50 constructed violation cases	Automated policy-engine test harness, run per build
Reliability (6.7)	Job orchestration logs	All jobs in evaluation period	Terminal-state audit; any job outside the four states is a defect

Metric	Data source	Sample	Method
Cost delta vs. multi-agent (6.6)	Evaluation harness	Systems B and C on shared benchmark (Section 28)	Cost-per-dataset comparison at matched task coverage

Product metrics (this section) and research metrics (Section 29) share the same measurement infrastructure and benchmark set.

7. Scale & Performance Requirements

Parameter	Target
File size	≤2 GB
Rows	≤10 million
Columns	≤500
Concurrent jobs	5
Maximum job duration	30 min
Maximum autonomous retries	3
LLM calls per issue (single-agent default)	1 proposal call, plus retry calls per Section 24

8. Cost Requirements

LLM usage must not scale naively with dataset size. The system uses Dataset → Deterministic profiler → Compressed evidence → single agent, never Dataset → LLM directly.

Cost controls: token budgets, evidence compression, caching, deterministic handling for obvious problems (e.g., exact duplicates never reach the agent), model selection by task complexity, and — as a structural cost control rather than a tuning knob — defaulting to one LLM call per issue instead of a specialist-plus-Judge chain.

Reported per job: cost per dataset, cost per million rows, cost per detected issue, cost per successful transformation.

9. Competitive / Alternative Analysis

9.1 Positioning

AutoCleanAI should not claim that existing tools cannot clean data. The differentiation is controlled AI reasoning over ambiguous transformations combined with deterministic execution, policy enforcement, validation, and measurable preservation of the original data — now delivered through a single, evidence-grounded reasoning agent rather than an LLM-to-LLM negotiation.

9.2 Comparable products

Category	Example products	What they do well	Where AutoCleanAI differs
Deterministic validation frameworks	Great Expectations, dbt tests	Precise, reproducible checks against known rules	Cannot reason about ambiguous, undefined-rule remediation
Interactive data prep	OpenRefine	Strong manual/semi-automated transform tooling with human in the loop	No AI-driven candidate generation or independent validation layer
Enterprise data prep platforms	Trifacta (Alteryx), Talend	Broad connector support, visual pipelines	Rule/heuristic-driven suggestions, not policy-gated LLM proposals with sandboxed execution
Agentic data-quality/observability	Monte Carlo, Soda, Acceldata, Bigeye, DQLabs	Monitoring, anomaly detection, alerting at scale, some agentic remediation suggestions	Focused on observability/alerting rather than sandboxed, validated, provenance-tracked repair execution
LLM-based data assistants	General “chat with your CSV” tools	Flexible natural-language interaction	Typically give the LLM direct or near-direct data access; no IR, sandbox, or independent validator

9.3 Closest research prior art

Cocoon, CleanAgent, HoloClean, LLMClean/RTClean, and AutoDCWorkflow each address pieces of this problem — semantic rule generation, statistical error detection, deterministic probabilistic repair, or LLM-generated cleaning workflows. None of these, as far as current literature shows, combine a policy-gated intermediate representation, sandboxed deterministic execution, and an independently-scored validator with an explicit unnecessary-modification metric and a designed “no safe transformation found” outcome.

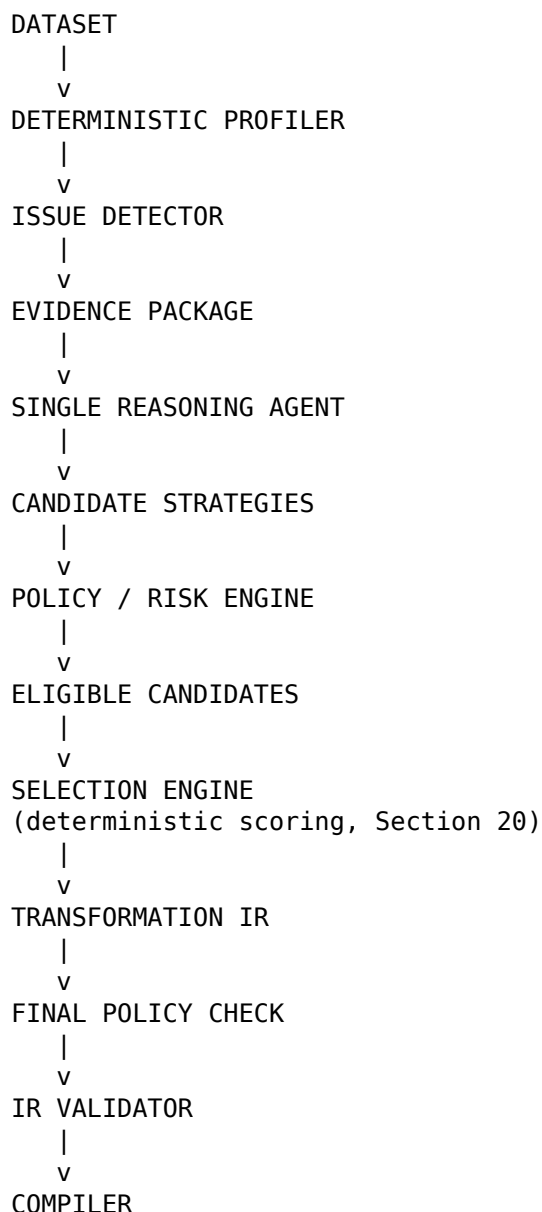
9.4 Evidence on multi-agent debate for this task

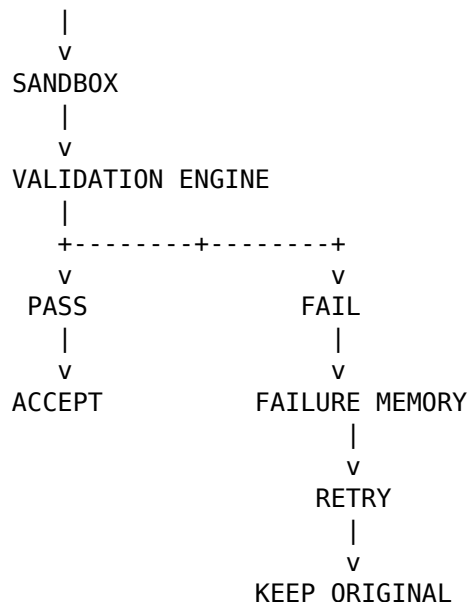
A 2026 factorial study on multi-agent debate for data cleaning, across three benchmarks, four model families, and over 6,000 task-condition pairs, found that debate degraded generation quality by 1.6 to 15.5 percentage points across all four tested models through critique-induced confusion, while improving error *detection* by roughly 27 percentage points F1. Self-verification (an agent checking its own work) failed to help at all; only an adversarially separated, tool-grounded critic produced a modest net improvement (+5.3pp) on generation, and even that required careful design. This is the direct evidence basis for Section 0.2’s decision to make single-agent the default and to keep multi-agent as an explicitly tested, not assumed, variant.

10. Product Differentiation

1. **Evidence-first AI** — the agent reasons over profiler-generated evidence, not raw data by default.
 2. **Structured transformation IR** — the LLM cannot arbitrarily execute Python.
 3. **Single-agent efficiency with optional multi-agent comparison** — one reasoning call per issue by default, benchmarked against a multi-agent variant rather than assumed to need one.
 4. **Policy-constrained selection** — only policy-eligible candidates ever reach execution.
 5. **Independent validation** — execution success does not imply semantic correctness.
 6. **Preservation-first philosophy** — unnecessary modification is explicitly measured.
 7. **Safe refusal** — “no safe transformation found” is a valid, designed outcome.
-

11. Core Architecture





What changed from the multi-agent design: the Missing/Outlier/Consistency specialist agents and the separate Judge agent are replaced by one reasoning agent that receives the full evidence package for an issue and returns ranked candidate strategies directly. The deterministic Selection Engine — not a second LLM call — makes the final pick among policy-eligible candidates, using the same configurable weighted scoring described in Section 20. This removes two LLM-to-LLM handoffs (specialist → Judge, Judge → IR) without removing any deterministic safety component.

12. Authority Boundary

Two authorities now govern the pipeline (reduced from three, since the Judge is removed):

- **Policy Engine** — determines *what is allowed?* Filters candidates before they can be selected, and re-checks the final IR before compilation.
- **Validation Engine** — determines *did the resulting transformation satisfy execution, constraint, and preservation requirements?*

The single reasoning agent proposes; it has no execution authority and no ability to override policy or validation outcomes. The deterministic Selection Engine (Section 20) — not the agent — makes the final choice among eligible candidates, which further reduces the agent's influence over outcomes compared to a Judge-based design. This separation is fundamental and, if anything, simpler to reason about and audit than the three-authority version.

13. AI Failure Handling

LLM failure is a normal system condition: timeout, API failure, rate limit, malformed response, invalid JSON, invalid IR, unsupported operation, low-confidence response.

Recovery hierarchy: LLM failure → Retry same request → Alternative model / provider → Deterministic fallback → Mark unresolved.

With a single agent per issue, there is one fewer point of failure than in the multi-agent design (no specialist-then-Judge chain where either link can fail). If the agent fails for an issue,

deterministic strategies (KEEP, FLAG, MEDIAN, MODE) may still be available depending on policy. The system must never fail the entire dataset merely because one LLM call failed.

14. AI Response Validation

Every LLM response passes through: LLM → JSON Schema Validation → IR Validation → Operation Allowlist → Policy Check. Malformed output is rejected. No malformed response reaches the compiler.

15. Data Privacy Architecture

Default: the agent receives aggregated statistics, schema information, issue summaries, distribution information, and limited anonymized examples where necessary. Raw rows are not sent to the LLM by default.

If raw samples are required: Need identified → Privacy policy check → Sampling / masking → Audit event → LLM. A single-agent design also means raw-data exposure risk exists at one call site per issue instead of several, simplifying the audit surface.

16. Sensitive Data Handling

Configurable data classifications: PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED.

Detection — two layers:

1. **Column-name / metadata heuristics (primary, fast):** pattern matching on column names (e.g., email, phone, address, income, government_id), schema metadata, user-provided classification.
2. **Content-based scanning (fallback, catches mislabeled columns):** sampled-value pattern detection (regex for emails, national IDs, phone numbers, card formats) and entropy/format scoring on a bounded sample of values per column, run regardless of column name, to catch cases like a column named notes containing sensitive values.

A column is treated as sensitive if either layer flags it. For RESTRICTED data, raw values must not be transmitted to external model providers unless explicitly permitted by deployment policy.

17. Access Control & Roles

Role	Upload/run jobs	View audit logs (own datasets)	View audit logs (org-wide)	Approve HIGH-risk transformations	Change policy config
Viewer	No	Yes	No	No	No
Contributor (default)	Yes	Yes	No	No	No
Approver	Yes	Yes	Yes	Yes	No
Admin	Yes	Yes	Yes	Yes	Yes

Datasets and jobs are scoped to a team/project namespace; cross-namespace access requires explicit, audited sharing. Pipeline mode additionally requires a service-identity path for non-interactive execution, distinct from human user roles.

18. Cloud Security

Provider-agnostic requirements: encryption at rest, TLS in transit, least-privilege access control, secrets management, audit logging of infrastructure access, short-lived credentials, defined dataset lifecycle/retention policies.

AWS-specific implementation (current default deployment): S3 bucket policies, IAM least privilege, private networking (VPC) where appropriate, AWS Secrets Manager, KMS, CloudWatch auditing. If a future deployment target is added, provider-agnostic requirements remain constant and each provider gets its own implementation subsection.

19. Approval Workflow & SLA

HIGH-risk transformations require an authenticated authorized user (Approver or Admin role).

Approval record must include: User ID, Timestamp, Transformation ID, Dataset version, Transformation reason, Policy version, Decision.

Pending-approval behavior:

- A job awaiting HIGH-risk approval enters a PENDING_APPROVAL state, tracked for the reliability metric (6.7), not a silent hang.
 - Requesting user and all Approver/Admin-role users in the namespace are notified when a job enters PENDING_APPROVAL.
 - Configurable approval timeout (default: 24 hours). On timeout, the job moves to UNRESOLVED with reason “approval timeout” — it does not auto-approve.
 - In Autonomous mode, a pending HIGH-risk item blocks only that transformation, not the rest of the job.
 - In Pipeline mode, a pending approval blocks promotion to the downstream target (Section 24).
-

20. Candidate Selection (Deterministic, Not a Second LLM Call)

Multiple candidates from the single agent may pass validation. The Selection Engine — not a Judge agent — picks deterministically among accepted candidates using configurable weights:

1. Validation success
2. Policy compliance
3. Risk level
4. Data preservation
5. Distribution preservation
6. Unnecessary modification minimization
7. Agent-reported confidence
8. Transformation complexity

Selection Score =
Validation Score
+ Preservation Score
+ Confidence
- Risk Penalty
- Modification Penalty
- Complexity Penalty

Weights are configurable and documented. The system must not simply choose “the candidate that improves the quality score the most.” Making this scoring deterministic rather than a second LLM judgment removes a full agent-to-agent handoff from the pipeline while preserving multi-candidate comparison.

21. Unnecessary Modification Rate

Primary safety metric: percentage of originally correct values or records unnecessarily modified. Where ground truth exists: $UMR = \text{unnecessarily modified correct values} / \text{total originally correct values}$. The system should aim to minimize UMR.

22. KEEP / FLAG / MODIFY

For every detected issue, the final decision is KEEP (no modification), FLAG (preserve original, mark the issue), or MODIFY (apply an approved transformation).

23. No-Change Final State

If no candidate passes acceptance criteria: KEEP ORIGINAL → mark issue unresolved → explain why. This is a valid, successful system outcome, not an error state.

24. Retry Mechanism and Termination

A failed transformation triggers replanning by the same single agent with updated evidence (failure reason, prior attempt, validation results) — not a hand-off to a different specialist. Previously failed strategies cannot be reused without new evidence.

Retry terminates when: maximum attempts reached; no unused candidate remains; no candidate satisfies acceptance criteria; improvement falls below the configured minimum threshold; or risk policy prevents remaining candidates. Then: NO SAFE TRANSFORMATION FOUND → RETAIN ORIGINAL → REPORT UNRESOLVED ISSUE.

25. Downstream Contract (Pipeline Mode)

Every dataset promoted to a downstream target carries its provenance metadata and final-dataset hash. Promotion is a distinct, logged step from “job completed” — a completed job

does not auto-promote unless explicitly configured. If a defect is discovered in an already-promoted dataset, AutoCleanAI can regenerate and re-promote a corrected version, but cannot retract data already consumed downstream — that is the downstream system’s own responsibility. Pipeline mode is out of scope for MVP sign-off.

26. Audit Immutability, Versioning, Compatibility, Rollback

Audit records are append-only, preserving original dataset hash, transformation hash, final dataset hash, and audit record. Every job records platform, profiler, agent, prompt, model, IR, policy, compiler, and dependency versions. Transformation IR is explicitly versioned (v1, v2, v3...); incompatible changes go through a migration layer or are reported as non-reproducible under the current runtime. Rollback operates at the dataset level: because the raw dataset is immutable, any transformation attempt can be discarded without modifying the original, and the user can revert to it at any time.

27. Quality Vector & Distribution Preservation

Quality metrics (Completeness, Validity, Consistency, Uniqueness) are kept distinct from preservation metrics (Row Retention, Distribution Preservation, Correlation Preservation, Modification Rate) rather than collapsed into one number. The underlying divergence metric (e.g., Wasserstein Distance) is always exposed alongside any derived score, and the scoring function is documented — a user never sees “94%” without knowing how it was calculated.

28. Research Evaluation

Because the shipped default is now single-agent, the evaluation framework exists specifically to test whether that choice is correct, not to justify a foregone multi-agent conclusion.

System	AI	Multi-Agent	Validation	Retry
A — Deterministic	No	No	Yes	No
B — Single-Agent (default)	Yes	No	Yes	No
C — Multi-Agent	Yes	Yes	Yes	No
D — Single-Agent + Retry	Yes	No	Yes	Yes

All systems share the same dataset, IR, compiler, sandbox, validator, acceptance policy, and metrics. Primary comparisons: A vs. B (value of LLM reasoning at all), B vs. C (whether multi-agent decomposition earns its added cost and latency over single-agent), B vs. D (value of retry/replanning on top of single-agent). System C is retained specifically so that if evidence later favors multi-agent for this task, the team has a ready comparison rather than having to build it from scratch.

29. Research Metrics

Detection: Precision, Recall, F1. **Repair:** Exact recovery accuracy where applicable, numeric repair error, categorical repair accuracy. **Decision quality:** Appropriate KEEP, appropriate FLAG, appropriate MODIFY, candidate selection accuracy. **Safety:** Unnecessary

Modification Rate, Data Loss Rate, Constraint Violation Rate, Unsafe Transformation Rejection Rate. **System:** Latency, LLM calls, Tokens, Cost, Retry count — reported per system (A/B/C/D) so the cost/latency case for single-agent is measured, not assumed. **Human evaluation:** Appropriateness rating, explanation usefulness, trust, time saved.

30. Real-World Evaluation

“Known labels” means externally established ground truth or independently validated annotations. If no such ground truth exists, the dataset is not used for exact recovery accuracy; instead: constraint satisfaction + human evaluation + preservation + unnecessary modification.

31. User Interface Requirements

The UI shows dataset overview, issues list, candidate strategies (with risk and expected modification counts), the decision made, and validation results (Execution / Constraints / Preservation / Policy) leading to a final ACCEPTED/REJECTED/UNRESOLVED state.

Surfacing uncertainty:

- When multiple candidates pass validation with selection scores within a configurable margin (default: within 10% of the top score), the UI shows the top 2-3 near-tied candidates side by side, not only the winner.
 - When the agent’s reported confidence is LOW even though the selected candidate was technically ACCEPTED, this is shown as a visible badge on that decision, not buried only in the audit log.
 - In Autonomous mode, any low-confidence decision is included in a post-run summary the user is prompted to review, even though it didn’t require approval to execute.
-

32. Deployment Modes

Mode 1 — Assisted: AI proposes → User approves → Execute. **Mode 2 — Autonomous:** Allowed LOW/MEDIUM transformations automatically execute; HIGH-risk operations require approval per Section 19’s SLA. **Mode 3 — Pipeline (future):** S3 → AutoCleanAI → Validated Dataset → Data Warehouse / ML Pipeline, subject to the downstream contract in Section 25.

33. Rollout Strategy

Stage 1 — Local Research Prototype: Python, Polars, local sandbox, LLM API. Goal: validate architecture. **Stage 2 — Cloud Prototype:** S3, ECS, Bedrock, simple API. Goal: validate cloud execution. **Stage 3 — Production-like MVP:** Add authentication, IAM, approval workflow, audit logging, monitoring, cost tracking, versioning.

34. Development Timeline

Assumptions: core team of 4 (2 backend/ML engineers, 1 data/infra engineer, 1 frontend engineer), with a research/eval lead shared part-time across Phases 7-8. The single-agent

default removes the specialist-plus-Judge integration work from v5.0's plan, shortening the AI track by roughly two weeks; that time is reallocated to buffer rather than removed from the timeline.

Track	Phase	Weeks	Notes
Data track	Profiler, issue detector, transformations	1-4	No LLM dependency; can start immediately
Execution track	IR schema, compiler, sandbox	3-6	Starts once profiler output shape is stable
AI track	Single-agent reasoning + evidence grounding	5-8	Depends on IR + evidence package being stable
Control track	Policy engine, deterministic candidate selection	7-10	Simpler than v5.0's Judge-based selection; can start once eligible-candidate contract is defined
Control track	Retry + failure handling	10-12	Depends on policy + validation engine
Eval track	Evaluation experiments incl. optional multi-agent variant (System C)	12-16	Depends on all systems (A/B/C/D) being runnable; System C is scoped as a research spike, not a shipping requirement
Platform track	AWS deployment, access control, approval SLA	14-18	Can overlap eval track
Frontend track	UI incl. uncertainty surfacing	7-19	Starts early on static views, iterates as backend stabilizes
Wrap	Final evaluation, DoD sign-off	19-20	Buffer included

This timeline covers Stages 1-3 and Modes 1-2. Pipeline mode and multi-format ingestion are out of this timeline and tracked post-MVP.

35. Major Risks

Risk	Severity	Mitigation
LLM hallucination	High	IR + validation
Data loss	High	Preservation constraints
Sensitive-data leakage	High	Evidence-first architecture + two-layer detection
Incorrect semantic repair	High	Human approval + validation

Risk	Severity	Mitigation
Single agent misses issues multi-agent decomposition would catch	Medium	Deterministic issue detector runs independently of the agent and covers known issue types regardless of agent output; System C comparison in Section 28 measures this gap directly
LLM outage	Medium	Retry/fallback
High cost	Medium	Evidence compression + budgets; single-agent default is itself a cost mitigation
Infinite retries	Medium	Termination rules
Model changes	Medium	Versioning
Dataset too large	Medium	Cloud workers / future distributed processing
Approval bottleneck stalls autonomous jobs	Medium	Approval SLA + timeout
Downstream consumption of a flawed promoted dataset (Pipeline mode)	Medium	Explicit downstream contract; Pipeline mode excluded from MVP

36. Final Definition of Done

Each item is marked **[Gating]** or **[Aspirational]**.

1. Upload a CSV. **[Gating]**
2. Receive deterministic EDA. **[Gating]**
3. See detected data-quality issues. **[Gating]**
4. Generate candidate strategies via the single reasoning agent. **[Gating]**
5. See why strategies were proposed. **[Gating]**
6. Have policy filter unsafe candidates. **[Gating]**
7. Generate a structured IR. **[Gating]**
8. Execute the transformation in a sandbox. **[Gating]**
9. Validate the result. **[Gating]**
10. Compare multiple valid candidates, including near-tied candidates surfaced per Section 31. **[Gating]**
11. Select the best valid candidate deterministically (no second LLM judgment). **[Gating]**
12. Retry failed transformations. **[Gating]**
13. Stop safely when no acceptable solution exists. **[Gating]**
14. Download the resulting dataset. **[Gating]**
15. View the complete transformation history. **[Gating]**
16. Reproduce a transformation using recorded versions. **[Gating]**
17. See estimated compute/LLM cost. **[Gating]**
18. Review/approve HIGH-risk transformations within the approval SLA. **[Gating]**
19. Verify that the raw dataset remains unchanged. **[Gating]**
20. Meet the UMR, unsafe-rejection, retention, and reliability targets in Section 6 on benchmark datasets. **[Gating]**
21. Meet the $\geq 50\%$ time-reduction and $\geq 70\%$ acceptance-rate targets. **[Aspirational — reported at launch, not a blocker]**
22. Access control and approval permissions enforced per Section 17 role model. **[Gating]**

23. Run System C (multi-agent) as a research spike against the shared benchmark and report B-vs-C results. **[Aspirational — informs post-MVP roadmap, not a launch blocker]**

Pipeline mode and non-CSV ingestion are explicitly excluded from this Definition of Done.

37. Final Product Philosophy

The project should not position itself as “*an AI that automatically cleans any dataset,*” nor as “*a multi-agent system for data cleaning.*” Both claims overstate what current evidence supports. AutoCleanAI’s claim is narrower and more defensible: within a bounded, measured, policy-constrained architecture, a single evidence-grounded reasoning agent, combined with deterministic execution and independent validation, can improve the selection of safe data-quality remediations over deterministic rules alone — and the system is designed to measure, not assert, whether further architectural complexity (such as multi-agent decomposition) is actually worth its cost on this task.